LAB 8 - AVL Trees

Data Structures and Algorithms Laboratory

Student Details

Name	Pratyush Deo Singh
Registration Number	25BCE5101
Semester	2nd
Subject	Data Structures and Algorithms
Slot	A2
Faculty Name	Dr. Malini A

Programs

Program 1: AVL Tree Insertion with Balance Factor

File Name:

q1_avl_insert_balance.c

Program Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    int height;
    struct Node* left;
    struct Node* right;
};

int height(struct Node* node) {
    if (node == NULL)
        return 0;
    return node->height;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

int getBalance(struct Node* node) {
    if (node == NULL)
        return 0;
    return height(node->left) - height(node->right);
}
```

```
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->height = 1;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node* rotateRight(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = 1 + max(height(y->left), height(y->right));
    x->height = 1 + max(height(x->left), height(x->right));

    return x;
}

struct Node* rotateLeft(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = 1 + max(height(x->left), height(x->right));
    y->height = 1 + max(height(y->left), height(y->right));

    return y;
}

struct Node* insert(struct Node* root, int value) {
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    else {
        printf("Duplicate value %d not allowed.\n", value);
        return root;
    }

    root->height = 1 + max(height(root->left), height(root->right));

    int balance = getBalance(root);

    if (balance > 1 && value < root->left->data)
        return rotateRight(root);

    if (balance < -1 && value > root->right->data)
        return rotateLeft(root);

    if (balance > 1 && value > root->left->data) {
        root->left = rotateLeft(root->left);
        return rotateRight(root);
```

```
    }

    if (balance < -1 && value < root->right->data) {
        root->right = rotateRight(root->right);
        return rotateLeft(root);
    }

    return root;
}

void inorderWithBalance(struct Node* root) {
    if (root == NULL)
        return;
    inorderWithBalance(root->left);
    printf("Node: %d | Balance Factor: %d\n", root->data, getBalance(root));
    inorderWithBalance(root->right);
}

int main() {
    struct Node* root = NULL;
    int n, value;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter %d values: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("\nInorder Traversal with Balance Factors:\n");
    inorderWithBalance(root);

    return 0;
}
```

Sample Test Input:

Enter number of nodes: 6

Enter 6 values: 10 20 30 40 50 25

Expected Output:

Node: 10 | Balance Factor: 0

Node: 20 | Balance Factor: 0

Node: 25 | Balance Factor: 0

Node: 30 | Balance Factor: 1

Node: 40 | Balance Factor: 0

Node: 50 | Balance Factor: 0

Output Screenshot:

```
Enter number of nodes: 6
Enter 6 values: 10 20 30 40 50 25

Inorder Traversal with Balance Factors:
Node: 10 | Balance Factor: 0
Node: 20 | Balance Factor: 0
Node: 25 | Balance Factor: 0
Node: 30 | Balance Factor: 0
Node: 40 | Balance Factor: -1
Node: 50 | Balance Factor: 0
```

Program 2: AVL Tree Deletion with Rotation Type and Height

File Name:

q2_avl_delete_rotation.c

Program Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    int height;
    struct Node* left;
    struct Node* right;
};

int height(struct Node* node) {
    if (node == NULL)
        return 0;
    return node->height;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

int getBalance(struct Node* node) {
    if (node == NULL)
        return 0;
    return height(node->left) - height(node->right);
}

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->height = 1;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

struct Node* rotateRight(struct Node* y) {
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = 1 + max(height(y->left), height(y->right));
    x->height = 1 + max(height(x->left), height(x->right));

    return x;
}

struct Node* rotateLeft(struct Node* x) {
    struct Node* y = x->right;
    struct Node* T2 = y->left;
```

```
    y->left = x;
    x->right = T2;

    x->height = 1 + max(height(x->left), height(x->right));
    y->height = 1 + max(height(y->left), height(y->right));

    return y;
}

struct Node* insert(struct Node* root, int value) {
    if (root == NULL)
        return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    else {
        printf("Duplicate value %d not allowed.\n", value);
        return root;
    }

    root->height = 1 + max(height(root->left), height(root->right));

    int balance = getBalance(root);

    if (balance > 1 && value < root->left->data)
        return rotateRight(root);

    if (balance < -1 && value > root->right->data)
        return rotateLeft(root);

    if (balance > 1 && value > root->left->data) {
        root->left = rotateLeft(root->left);
        return rotateRight(root);
    }

    if (balance < -1 && value < root->right->data) {
        root->right = rotateRight(root->right);
        return rotateLeft(root);
    }

    return root;
}

struct Node* findMin(struct Node* root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

struct Node* deleteNode(struct Node* root, int value) {
    if (root == NULL) {
        printf("Value %d not found.\n", value);
        return root;
    }

    if (value < root->data)
        root->left = deleteNode(root->left, value);
    else if (value > root->data)
        root->right = deleteNode(root->right, value);
```

```
    else {
        if (root->left == NULL || root->right == NULL) {
            struct Node* temp = root->left ? root->left : root->right;
            if (temp == NULL) {
                temp = root;
                root = NULL;
            } else {
                *root = *temp;
            }
            free(temp);
        } else {
            struct Node* temp = findMin(root->right);
            root->data = temp->data;
            root->right = deleteNode(root->right, temp->data);
        }
    }

    if (root == NULL)
        return root;

    root->height = 1 + max(height(root->left), height(root->right));

    int balance = getBalance(root);

    if (balance > 1 && getBalance(root->left) >= 0) {
        printf("Rotation performed: LL\n");
        return rotateRight(root);
    }

    if (balance > 1 && getBalance(root->left) < 0) {
        printf("Rotation performed: LR\n");
        root->left = rotateLeft(root->left);
        return rotateRight(root);
    }

    if (balance < -1 && getBalance(root->right) <= 0) {
        printf("Rotation performed: RR\n");
        return rotateLeft(root);
    }

    if (balance < -1 && getBalance(root->right) > 0) {
        printf("Rotation performed: RL\n");
        root->right = rotateRight(root->right);
        return rotateLeft(root);
    }

    return root;
}

void inorder(struct Node* root) {
    if (root == NULL)
        return;
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
}

int main() {
    struct Node* root = NULL;
    int n, value, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
```

```
    printf("Enter %d values: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        root = insert(root, value);
    }

    printf("Enter value to delete: ");
    scanf("%d", &key);

    root = deleteNode(root, key);

    printf("Inorder Traversal after deletion: ");
    inorder(root);
    printf("\n");

    printf("Updated Height of AVL Tree: %d\n", height(root));

    return 0;
}
```

Sample Test Input:

Enter number of nodes: 5

Enter 5 values: 30 20 40 10 25

Enter value to delete: 40

Expected Output:

Rotation performed: LL

Inorder Traversal after deletion: 10 20 25 30

Updated Height of AVL Tree: 3

Output Screenshot:

```
Enter number of nodes: 5
Enter 5 values: 30 20 40 10 25
Enter value to delete: 40
Rotation performed: LL
Inorder Traversal after deletion: 10 20 25 30
Updated Height of AVL Tree: 3
```